

# Python Language Coding Standards

Programming language: Python

## 1. File Header

At the top of every file, write a comment to describe the purpose of the file:

```
# file: example.py
# purpose: Show how to write clean Python code
# author: <Your Name>
# date: 2025-09-29
```

## 2. Functions

- Every module, class, and function must have a docstring describing purpose, arguments, and return values.
- Use triple quotes `"""` for docstrings.
- Comments should explain why, not just what.

```
def calculate_discount(price: float, rate: float) -> float:
```

```
    """
```

```
    Calculate discounted price.
```

```
    Args:
```

```
        price (float): Original price of the item.
```

```
        rate (float): Discount rate (e.g., 0.1 for 10%).
```

```
    Returns:
```

```
        float: Final price after discount.
```

```
    """
```

```
    return price * (1 - rate)
```

## 3. Comments

- Use comments only if the code is not obvious.
- Keep them short.

```
# loop through scores
for s in scores:
    total += s
```

## 4. Naming

- Variables and functions: snake\_case
- Classes: CamelCase
- Constants: UPPER\_CASE

```
MAX_SCORE = 100
```

```
class Student:  
    Pass
```

## 5. Formatting

- Indent with Tab.
- Keep lines short (less than 80 characters).
- Leave blank lines between sections.

## 6. Imports

- Standard library first, then external libraries, then local imports.

```
import math  
import sys
```

```
import requests
```

```
from utils import helper
```

## 7. Variable Naming Conventions

- Descriptive, concise, and unambiguous: user\_count, total\_bytes
- Booleans start with is\_, has\_, can\_, should\_: is\_active, has\_error
- Iteration variables should be meaningful unless trivially obvious: for user in users:
- Temporary variables okay for short scopes: i, j, k
- Avoid ambiguous names (data, info) unless the context is truly generic
- Do not shadow built-ins (list, dict, id, type)

```
is_valid_email = True  
retry_count = 3
```

## 8. Function & Method Naming

- Use verbs and be action-oriented: load\_config, compute\_total, send\_email

- Avoid long parameter lists (prefer small and composable functions).

## 9. Project & File Organization

Inside each module file, order sections as:

1. File introduction
2. Imports
3. Custom exceptions
4. Data classes / models
5. Function